

Lab 12 — Low Power: Deep Sleep and Waking Up

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2 hrs
Introduces	power domains, RTC memory, wake causes
Prerequisites	Labs 1 and 10.
Beyond the board	Nothing. A current meter is useful but not required.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§12.1 objective**, **§12.2 parts and wiring**, **§12.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

12.1 Objective & TM4C bridge

Make the chip do a short burst of work and then sleep at microamps, waking on a timer. This is the pattern behind every battery-powered sensor, and it forces a genuinely different way of structuring a program.

TM4C BRIDGE

The TM4C had sleep modes too, and you may have read about them. What is different here is the consequence: on the ESP32, waking from deep sleep is a **reboot**, not a resume. Your program restarts from the top of `app_main` with RAM wiped. That single fact reshapes the whole design, and it has no real TM4C analogue.

12.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	nothing external	—
Multimeter in current mode (optional)	nice to have	The only way to actually <i>see</i> the power saving.

NOTE**On measuring the current for real**

The log tells you the chip slept; it does not tell you it slept *efficiently*. To measure, you break the supply line and put an ammeter in series. Be aware that a dev board will never reach datasheet sleep figures — the USB-serial chip, the regulator and the power LED all keep drawing, often more than the ESP32 itself. Expect milliamps from a dev board where a bare module would show microamps. Knowing *why* the board does not match the datasheet is itself a good piece of engineering judgement to be able to explain.

12.3 How it works

Everything so far assumed the board is plugged in. This lab is about what changes when it is running from a battery, and the answer reshapes how you structure a program.

The arithmetic that forces the whole design

Start with a number, because it makes the rest obvious. A typical coin cell holds about 220 mAh. Your board in normal operation, radio idle, draws roughly 40 mA.

$220 \div 40 \approx \mathbf{5.5 \text{ hours}}$. That is your entire battery life if the chip simply stays awake. No firmware cleverness recovers from that; the only lever is *not being awake*.

Now suppose the device wakes for 200 ms every 10 seconds, drawing 40 mA while awake and 10 μ A while asleep. The **average** current is:

$$(40 \text{ mA} \times 0.2 \text{ s} + 0.01 \text{ mA} \times 9.8 \text{ s}) / 10 \text{ s} \approx 0.81 \text{ mA}$$

$220 \div 0.81 \approx \mathbf{270 \text{ hours}}$, about eleven days — roughly a fiftyfold improvement, from the same hardware, purely by being asleep 98% of the time. That ratio is why deep sleep exists and why battery devices are built the way they are.

The key idea in that calculation is **average current**. Peak current barely matters; what drains the cell is the area under the curve. So the design question is never "how do I make the chip use less power" but "how do I spend less time awake."

Power domains: the chip is not one switch

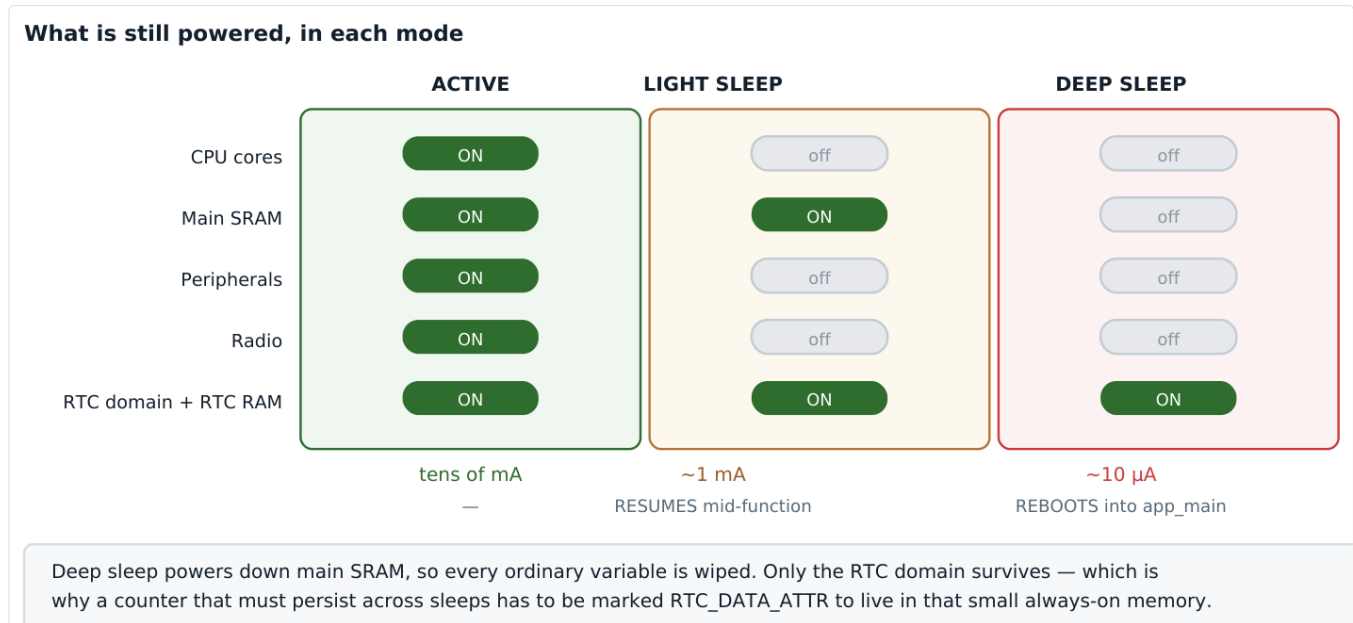


Figure 12.1. Sleep modes are combinations of which power domains stay energised. Deep sleep keeps only the always-on RTC domain, which is why main RAM is wiped and why waking is a reboot rather than a resume.

Which parts of the chip stay powered in each mode.

Domain	Active	Light sleep	Deep sleep
CPU cores	ON	off	off
Main SRAM	ON	ON	off
Peripherals	ON	off	off
Radio	ON	off	off
RTC domain + RTC RAM	ON	ON	ON

	Active	Light sleep	Deep sleep
Current	tens of mA	~1 mA	~10 microamps
On wake	-	RESUMES mid-function	REBOOTS into <code>app_main</code>

Deep sleep powers down main SRAM, so every ordinary variable is wiped. Only the RTC domain survives, which is why a counter that must persist across sleeps has to be marked `RTC_DATA_ATTR` to live in that small always-on memory.

The silicon is divided into regions that can be powered independently: the CPU cores, the main SRAM, the peripherals, the radio, and a small **RTC domain** that contains a low-power oscillator, a timer, a little memory, and enough logic to wake everything else back up.

The three modes are just choices about which of those to leave on.

	Active	Light sleep	Deep sleep
CPU	Running	Paused	Powered off
Main SRAM	On	Retained	Lost
Peripherals	On	Mostly off	Off
Current	tens of mA	~1 mA	~10 μ A
On wake	—	Resumes the next line	Reboots into <code>app_main</code>

Light sleep is the convenient one: RAM survives, execution picks up where it left off, and your program structure does not change at all. But it is a hundred times more expensive than deep sleep, which is the difference between eleven days and three hours.

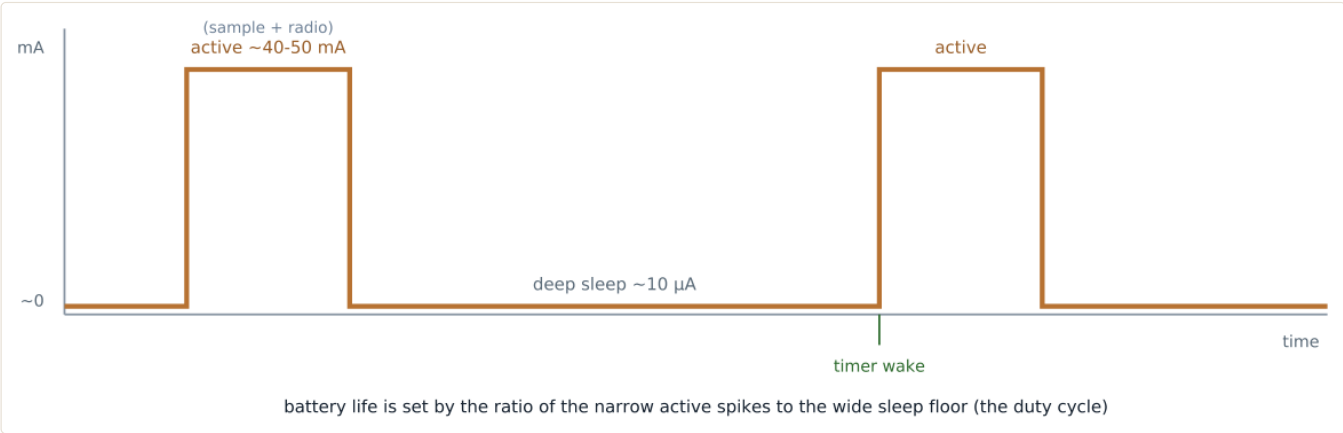


Figure 12.2. The duty cycle in the time domain: a brief high-current burst, then a long stretch at microamps. It is the *area under this curve* that drains the battery, which is why shortening the burst matters more than reducing the peak.

Current draw over time across an active burst and deep sleep.

The trace shows narrow, tall spikes of active current (sampling plus radio, roughly 40 to 80 mA) separated by long flat stretches at the deep-sleep floor of about 10 microamps. Each spike is followed by a timer wake that begins the next one.

Battery life is set by the **RATIO** of the narrow active spikes to the wide sleep floor. That ratio is the duty cycle, and it is the **area under the curve**, not the peak height, that drains the cell.

Waking from deep sleep is a reboot — and everything follows from that

This is the fact that catches people, so state it plainly: `esp_deep_sleep_start()` **does not return**. The chip powers down. When the wake source fires, it goes through reset, the bootloader runs, and `app_main` is called from the top exactly as on a cold start. There is no "resuming" and no stack to come back to.

Two consequences you must design around:

Every ordinary variable is gone. Globals, statics, the heap, the stack — all in main SRAM, all wiped. A counter declared normally will read 0 on every single wake.

You must ask why you started. Because the entry point is the same for a cold boot and a wake, the first thing a low-power program does is check the **wake cause** and branch: first-time setup versus continue the cycle. **TM4C bridge:** the nearest equivalent is reading a reset-cause register to tell a watchdog reset from a power-on reset, then acting differently.

RTC memory: the small thing that survives

If all RAM is wiped, how does a device remember how many times it has woken, or where it was in a sequence? The RTC domain contains a few kilobytes of memory that stays powered through deep sleep. Marking a variable `RTC_DATA_ATTR` places it there instead of in ordinary RAM.

That gives you the third tier of the persistence ladder from §10.3, and choosing between them is a real decision:

Storage	Survives deep sleep	Survives power loss	Write cost	Wears out?
Ordinary RAM	No	No	Free	No
RTC RAM	Yes	No	Free	No
NVS / flash	Yes	Yes	Slow	Yes

Using RTC RAM for a wake counter rather than NVS is deliberate: it is instant, and it does not consume flash endurance on every single wake. Losing the count when the battery is changed is an acceptable trade. The pattern that combines them — count in RTC RAM, write a summary to NVS occasionally — is the one real products use, and it is the exercise in §12.7.

Wake sources

A timer is only one option. You arm whichever sources make sense and the first to fire wins:

- **Timer** — wake after a fixed interval. What this lab uses.
- **GPIO / EXT0 and EXT1** — wake when a pin changes. A button, or a sensor's interrupt line saying "I have data."
- **Touch pad** — wake on capacitive touch.
- **ULP coprocessor** — a tiny low-power processor that can run simple monitoring code while the main cores sleep, and wake them only when something interesting happens. This is how a device can watch a sensor continuously and still last months.

Only pins in the RTC domain can serve as GPIO wake sources, which is a genuine constraint on your pin assignment when you get to a real battery design.

WARNING**Why your measured current will not match the datasheet**

The datasheet says roughly 10 μA in deep sleep. You will measure milliamps on a dev board, and the board is not faulty. The USB-to-serial bridge chip, the 3.3 V regulator's quiescent draw, and the power LED are all still running — and together they draw far more than the sleeping ESP32 does. Reaching datasheet figures needs a bare module on a board designed for it.

Understanding *why* the number disagrees is more valuable than the number. It is a real engineering observation: on a low-power design, the supporting components often dominate the budget, and the first thing a power-conscious hardware designer removes is the always-on LED.

12.4 Procedure & code

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_sleep.h"
#include "esp_log.h"

#define SLEEP_US (5 * 1000000ULL)           // 5 seconds, in microseconds

static RTC_DATA_ATTR int wake_count = 0;    // RTC RAM -> survives deep sleep

void app_main(void) {
    esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
    if (cause == ESP_SLEEP_WAKEUP_TIMER)
        ESP_LOGI("lab12", "woke from deep sleep; wake #%d", ++wake_count);
    else
        ESP_LOGI("lab12", "cold boot (power-on or reset)");

    // --- your burst of real work goes here: sample, compute, transmit ---
    vTaskDelay(pdMS_TO_TICKS(500));

    ESP_LOGI("lab12", "sleeping for 5 s");
    esp_sleep_enable_timer_wakeup(SLEEP_US);
    esp_deep_sleep_start();                  // does NOT return -- reboots into app_main on wake
}
```

12.5 Code walkthrough, line by line

Twenty-three lines, and two of them do things nothing earlier in the manual has done.

Line 3 · `#include "esp_sleep.h"`

The sleep API: wake-source configuration, the sleep entry functions, and the wake-cause query. Note it lives under **System** in the documentation rather than Peripherals, because sleep is a whole-chip state rather than one piece of hardware.

Line 6 · `#define SLEEP_US (5 * 1000000ULL)`

Five seconds expressed in microseconds, because the sleep API counts microseconds. Three details.

The **parentheses** around the expression matter for any macro that expands to arithmetic. Without them, `SLEEP_US / 2` would expand to `5 * 1000000 / 2` — which happens to be right here, but the habit protects you when it is not.

The **ULL** suffix makes the literal an **unsigned long long**. Five million fits in a 32-bit int, but the sleep function takes a 64-bit microsecond count, and a longer sleep — an hour is 3.6 billion microseconds — would overflow 32 bits silently. Matching the literal's width to the parameter is the same discipline as **1ULL** in the pin masks.

Written as `5 * 1000000` rather than `5000000` so the magnitude is readable at a glance; the compiler folds it at compile time.

Line 8 · `static RTC_DATA_ATTR int wake_count = 0;`

The line that makes the whole demonstration possible. **RTC_DATA_ATTR** is a placement attribute: it tells the linker to put this variable in the RTC domain's memory rather than in ordinary SRAM, so it survives deep sleep.

Delete the attribute and the program still compiles and runs — and prints "wake #1" forever, because the variable is wiped and re-initialised on every wake. That is a good thing to try deliberately.

NOTE

The initialiser runs only on a cold boot

Worth pausing on, because it looks like it should reset every time. `= 0` is not executed as a statement; it is startup code that populates the variable's memory before `app_main` runs. On a deep-sleep wake, the RTC memory was never powered down, and the startup code deliberately skips re-initialising that region. So the value carries forward. That is exactly the behaviour you want, and it is why the counter climbs.

RTC memory is small — a few kilobytes shared with the ULP coprocessor — so it is for state, not for buffers.

Line 11 · `esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();`

Asks the chip why it started. This is the **first** thing a deep-sleep program does, for the reason in §12.3: the entry point is identical for a cold boot and a wake, so the code has to distinguish them itself.

The return type is an enum with values including `ESP_SLEEP_WAKEUP_TIMER`, `ESP_SLEEP_WAKEUP_EXT0` and `EXT1` (GPIO), `ESP_SLEEP_WAKEUP_TOUCHPAD`, `ESP_SLEEP_WAKEUP_ULP`, and `ESP_SLEEP_WAKEUP_UNDEFINED` — that last one meaning "not a wake at all," which is what a genuine power-on or reset reports.

Lines 12-15 · the branch

`if (cause == ESP_SLEEP_WAKEUP_TIMER)` takes the "continuing the cycle" path; anything else is treated as a cold boot. In real firmware this branch is where you separate **one-time setup** — provisioning, calibration, connecting to a network for the first time — from the repeating work.

Note `++wake_count` is **pre-increment**, so the variable is incremented *before* its value is passed to the log call, and the first wake prints "#1" rather than "#0". Contrast with the post-increment `n++` in Lab 2, which printed the value before incrementing. Choosing between them here is a real one-character decision about what the log says.

Lines 17-18 · the work burst

The comment marks where a real device's actual job goes: wake, sample the sensor, compute, transmit, and get back to sleep. `vTaskDelay(500)` stands in for that.

In a real design this section is where you spend your battery, so every millisecond here matters. The 500 ms placeholder is deliberately generous — a genuine sensor read and a BLE advertisement can finish in far less, and shortening this is the single highest-leverage optimisation in a low-power product.

Line 21 · `esp_sleep_enable_timer_wakeup(SLEEP_US);`

Arms the wake source. It does not sleep — it configures the RTC timer so that when the chip does sleep, it will be woken after this interval.

You can arm several sources before sleeping and the first to fire wins, which is how a device sleeps until *either* a timer expires *or* the user presses a button. Arming nothing at all is a real mistake: the chip sleeps forever and only a physical reset recovers it.

Line 22 · `esp_deep_sleep_start();`

Enters deep sleep, and **does not return**. Its declared type is `void`, but the real contract is stronger than that: control never comes back to this function. Any code you write after it is unreachable, and the compiler may warn you so.

Everything shuts down: cores, main RAM, peripherals, radio. Only the RTC domain stays alive, watching for the wake source you armed. When it fires, the chip resets and `app_main` runs again from line 10.

Before this line, the driver flushes pending work — but not always your serial output, which is why a log line printed immediately before sleeping can be truncated. Adding a short delay before sleeping is a common practical fix.

Line 23 · `}`

`app_main` technically ends here, but execution never reaches it. The cycle is: run from line 10, sleep at line 22, wake, run from line 10 again. That loop — a program whose "loop" is implemented by rebooting — is genuinely different from every other lab in this series, and it is the shape almost every battery-powered product takes.

12.6 Docs & references

DOCS

Trace every line to its source

`esp_sleep_enable_timer_wakeup`, `esp_deep_sleep_start`, `esp_sleep_get_wakeup_cause` — API Reference → System → **Sleep Modes**. Read the full list of wake sources; GPIO and touch wake are the ones you would use in a real product.

`RTC_DATA_ATTR` and RTC memory size — the same Sleep Modes page, plus API Guides → Memory Types. Find out how many bytes of RTC RAM you actually get.

Current consumption figures — ESP32-S3 **Datasheet**, the current-consumption table. Compare the deep-sleep figure there with what you measure on a dev board and understand the discrepancy.

Light sleep — same page. Worth reading the comparison to understand when resume-in-place beats reboot.

12.7 Verify, troubleshoot, your turn

Verify. The first run logs "cold boot." Every five seconds after that it logs "woke from deep sleep; wake #N" with N climbing. Press RESET and it says "cold boot" again with the counter reset — because a reset cycles the RTC domain too, which is exactly the distinction RTC memory draws.

Troubleshoot

Symptom	Cause and fix
Counter always shows 1	Missing <code>RTC_DATA_ATTR</code> .
Always reports cold boot	Something is resetting the chip rather than waking it. Check for a brownout in the log.
Monitor disconnects on sleep	Expected on some boards — the USB peripheral powers down. Reconnect, or use the UART port.
Never wakes	No wake source was armed before <code>esp_deep_sleep_start()</code> .
Measured current is milliamps, not microamps	Normal for a dev board. See the note in §12.2.

Your turn

1. **Add a GPIO wake source** so the BOOT button wakes the chip early, and log which source caused each wake. Now you have a device that sleeps until either a timer or a user acts, which is how most real products behave.
2. **Combine with Lab 10.** Keep a fast counter in RTC memory, and write a summary to NVS only once every hundred wakes. That is a real engineering pattern: use cheap volatile storage for frequent updates and pay the flash cost rarely, protecting endurance.
3. **Estimate battery life.** Measure or look up the active and sleep currents, and compute how long a 2000 mAh cell would last at your duty cycle. Then change the sleep time and see how the answer

moves. That calculation is the entire argument for deep sleep, and being able to do it out loud is a strong signal in an interview.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 12.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12 ←	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.